

HANDS-ON LAB

Git Hands-On Lab

Git Configuration, Notepad++ Integration & Your First Repository

TRACK	Version Control Fundamentals
TOPICS COVERED	Git configuration · Default editor setup · init / add / commit / push / pull
ESTIMATED TIME	30 minutes

Objectives

Become familiar with core Git commands: **git init**, **git status**, **git add**, **git commit**, **git push**, and **git pull**.

In this hands-on lab, you will learn how to:

- Set up your machine with Git configuration
- Integrate notepad++.exe with Git and make it the default editor
- Add a file to a source code repository

Prerequisites

- Install the Git Bash client on your machine

Note: Please follow the steps below to create a free account on GitHub/GitLab. Do **not** use company credentials to log in. Estimated time to complete this lab: **30 minutes**.

Step 1 — Set Up Your Machine with Git Configuration

To create a new repository, sign up with GitLab and register your credentials. Log in to GitLab and create a project named “**GitDemo**”.

1. Check that the Git client is installed properly

```
git --version
```

If the output shows Git along with its version information, the Git client is installed properly.

2. Configure your user-level Git identity (name and email)

```
git config --global user.name "Your Name"
git config --global user.email "youremail@example.com"
```

3. Verify the configuration is set correctly

```
git config --global --list
```

You can also check individual values:

```
git config --global user.name
git config --global user.email
```

Step 2 — Integrate notepad++.exe as the Default Git Editor

1. Check if notepad++.exe runs from Git Bash

```
notepad++
```

If Git Bash responds with something like `bash: notepad++: command not found`, notepad++.exe has not been added to the environment PATH variable.

To add the path of notepad++.exe to the environment variable:

- Go to **Control Panel** → **System** → **Advanced System Settings**
- Go to the **Advanced** tab → **Environment Variables**
- Under User variables, select **Path** → click **Edit** → **New** → add the Notepad++ install folder, e.g. `C:\Program Files\Notepad++`

2. Exit and reopen Git Bash, then execute

```
notepad++
```

Notepad++ should now open from the Git Bash shell.

3. Create an alias command for notepad++.exe

Open the bash profile in notepad++:

```
notepad++ ~/.bash_profile
```

Add the following line, then save and close the file:

```
alias notepad++="/c/Program\ Files/Notepad++/notepad++.exe"
```

Reload the profile so the alias takes effect:

```
source ~/.bash_profile
```

4. Configure Notepad++ as the Git editor

```
git config --global core.editor "'C:/Program Files/Notepad++/notepad++.exe' -multiInst -notabbar -
```

5. Verify that Notepad++ is the default editor

```
git config --global -e
```

The `-e` option stands for **editor**. This command opens the entire global Git configuration inside Notepad++, confirming it is set correctly.

Step 3 — Add a File to a Source Code Repository

1. Create a new project folder “GitDemo” and initialize it

```
mkdir GitDemo
cd GitDemo
git init
```

Git Bash initializes the “GitDemo” repository.

2. Verify initialization by viewing hidden files

```
ls -la
```

This displays all hidden files (including the `.git` folder) in the Git working directory.

3. Create a file “welcome.txt” and add content to it

```
echo "Welcome to Git Hands-On Lab" > welcome.txt
```

4. Verify that “welcome.txt” was created

```
ls -la
```

5. Verify the file's content

```
cat welcome.txt
```

6. Check the status of the working directory

```
git status
```

The file “welcome.txt” now appears as an **untracked file** in the Git working directory.

7. Stage the file so it is tracked by the Git repository

```
git add welcome.txt
```

8. Add a multi-line commit message using the default editor

```
git commit
```

Since Notepad++ is configured as the default editor, it opens automatically so you can type a multi-line commit message, for example:

```
Add welcome.txt
```

```
Initial commit for GitDemo project.  
Includes a simple welcome message file.
```

Save and close the Notepad++ window — Git Bash then completes the commit using that message.

9. Confirm local and working directory are in sync

```
git status
```

The output should show nothing to commit, working tree clean. “welcome.txt” is now added to the local repository.

10. Create the remote repository

Sign up on GitLab (use a personal, non-company email) and create a new remote project named “GitDemo”. Then link your local repository to it:

```
git remote add origin https://gitlab.com/<your-username>/GitDemo.git
```

11. Pull the remote repository

```
git pull origin master
```

12. Push the local repository to the remote

```
git push origin master
```

Practical notes: If your local default branch is “master” but GitLab creates “main”, rename it first with `git branch -M main` and adjust the pull/push commands accordingly. On first push you will be prompted to authenticate — GitLab now requires a **personal access token** instead of your account password for HTTPS pushes.